

On the Expressive Power of Languages for Static Variability – Summary

Paul Maximilian Bittner ^{1,2}, Alexander Schultheiß ³, Benjamin Moosherr ², Jeffrey M. Young ⁴, Leopoldo Teixeira ⁵, Eric Walkingshaw ⁶, Parisa Ataei ⁷, and Thomas Thüm ¹

Abstract: We report about recent research on the foundations of software product-line engineering, originally published at the OOPSLA issue of the Proceedings of the ACM on Programming Languages (PACMPL) in 2024 [Bi24]. Our research addresses fundamental open problems in software engineering of configurable systems with programming language theory.

Variability permeates software development to satisfy ever-changing requirements and mass-customization needs. A prime example is the Linux kernel, which employs the C preprocessor to specify a set of related but distinct kernel variants. To study, analyze, and verify variational software, several formal languages have been proposed. For example, the choice calculus has been successfully applied for type checking configurable software, while other formalisms have been used for variational model checking or change impact analysis. Yet, these languages have not been formally compared, hence, little is known about their relationships. Crucially, it is unclear to what extent one language subsumes another, how research results from one language can be applied to other languages, and which language is suitable for which purpose or domain. In our OOPSLA paper, we propose a formal framework to compare the expressive power of languages for static (i. e. compile-time) variability. After formalizing and unifying existing languages, we can formally compare these languages based on mutual compilers and a meta-theory on expressiveness. We find different levels of expressiveness, with the choice calculus being among the most expressive languages.

Keywords: variation, software product lines, configuration, language semantics

1 Summary

Explicit variability in software is a reoccurring phenomenon across many areas in science and business. Variability means that a system or piece of information may emerge in similar but different *variants* from a common set of underlying atomic elements. For example, the source code of the Linux kernel is variational because it can *and must* be configured to a particular kernel variant *before* a kernel can be used. While the Linux kernel consists of a

¹ TU Braunschweig, Germany, paul.bittner@uni-ulm.de,  <https://orcid.org/0000-0001-9388-0649>; thomas.thuem@tu-braunschweig.de,  <https://orcid.org/0000-0001-8069-9584>

² Ulm University, Germany, paul.bittner@uni-ulm.de,  <https://orcid.org/0000-0001-9388-0649>; benjamin.moosherr@uni-ulm.de,  <https://orcid.org/0009-0004-2670-8127>

³ University of Bern, Switzerland, alexanderschultheiss@pm.me,  <https://orcid.org/0000-0002-1509-1449>

⁴ Epic Games, USA, jeffrey.young@epicgames.com,  <https://orcid.org/0000-0002-2935-772X>

⁵ Federal University of Pernambuco, Brazil, lmt@cin.ufpe.br,  <https://orcid.org/0000-0002-6154-1666>

⁶ Unaffiliated, eric@walkingshaw.net,  <https://orcid.org/0000-0003-1214-0791>

⁷ Shielded Technologies LLC, USA, parisa.ataei@shielded.io,  <https://orcid.org/0000-0002-6703-2360>

single code base, its multiple thousand configuration options impose many different kernel variants, in fact so many, that the number of variants cannot be computed for newer versions.

To model and analyze implementations of configurable systems, numerous formal variability languages have been proposed. For instance, many languages depict variability in terms of a choice $F\langle e_{then}, e_{else} \rangle$ between alternative expressions e_{then} and e_{else} , where the annotation F denotes a feature whose inclusion or exclusion determines which alternative to pick. Compositional languages model variability in terms of distinct modules, each representing a unique feature, that may be composed in different combinations.

With different paradigms for variability, a zoo of languages, and different syntactical constructs available, several questions remain unanswered. Which language should be picked for new research efforts or tools? Do research results based on one language also apply to other languages? Can all languages express the same sets of variational systems? To the best of our knowledge, a formal characterization and comparison of variability languages has not yet been attempted, and discussions remain scarce, mostly informal, or brief.

We begin the journey to answer these questions by introducing a formal framework for comparing variability languages to guide researchers, practitioners, and language designers. The key idea is to identify the semantic domain of each language and compare these domains based on the fundamental yet unexplored properties of completeness, soundness, and expressiveness. Completeness serves as a lower bound (i. e., a language can express at least a given semantic domain), soundness serves as an upper bound (i. e., a language can express at most a given semantic domain), and expressiveness serves as a relative comparison (i. e., a language can express at least the semantic domain of another language). Based on these properties, we explore the relationships of existing languages and excavate a hierarchy of expressiveness. The landscape of languages we discovered bridges the gaps between parallel and so far independent research efforts.

2 Data Availability Statement

All our contributions are formalized in the programming language and proof-assistant Agda. Our library is open source on Github and archived on Zenodo [25]. At OOPSLA'24, our library received the ACM Artifacts Available , ACM Artifacts Evaluated Reusable , and ACM Artifacts Results Reproduced  badges and won a *distinguished artifact award*.

References

- [25] Vatrás – Agda Library to Study the Expressive Power of Variability Languages, Zenodo DOI: 10.5281/zenodo.13502454, Github: <https://github.com/VariantSync/Vatras>, 2025.
- [Bi24] Bittner, P. M.; Schultheiß, A.; Moosherr, B.; Young, J. M.; Teixeira, L.; Walkingshaw, E.; Ataei, P.; Thüm, T.: On the Expressive Power of Languages for Static Variability. Proceedings of the ACM on Programming Languages (PACMPL) (OOPSLA2), 2024.